

Software Design & Architecture Project

Michiel Brand

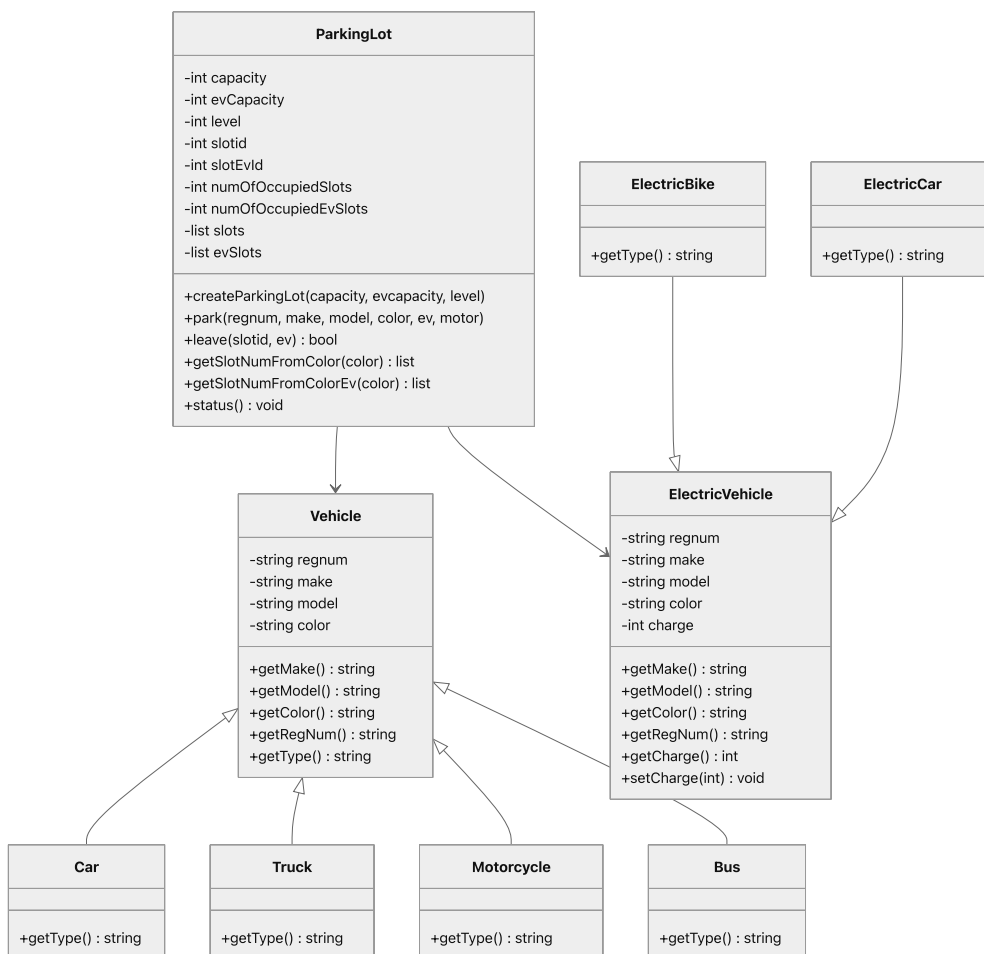
Q173978195964068764

26 October 2025

Original Design - UML Diagrams

This document contains UML diagrams representing the original parking lot manager architecture before refactoring.

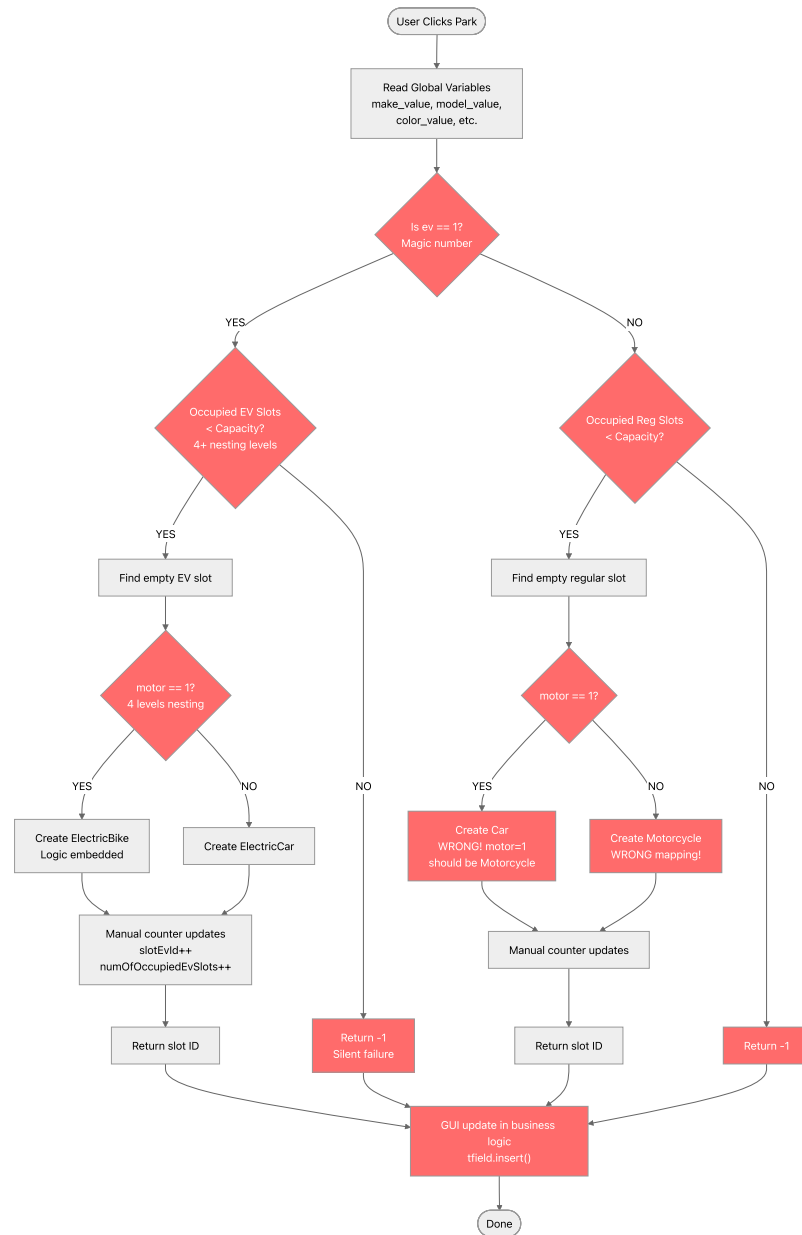
Structural UML Diagram (Class Diagram) - Original Design



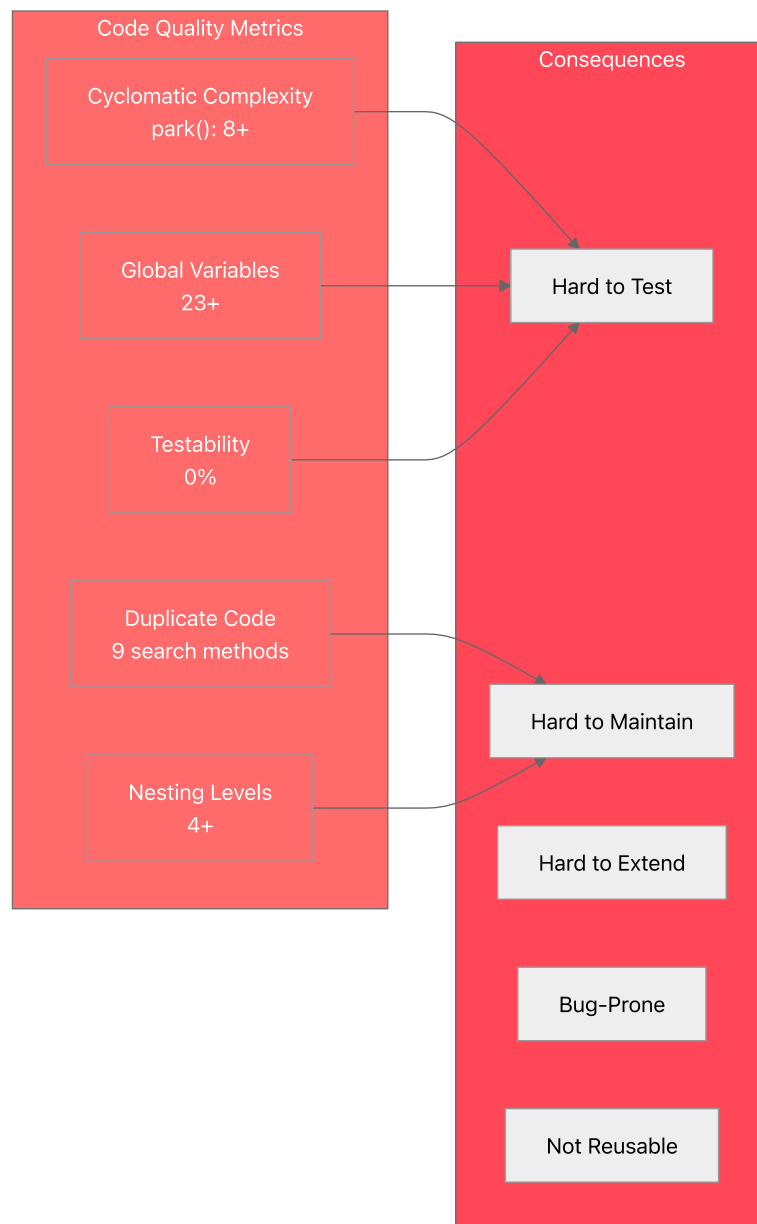
Issues Summary

Code Smell	Impact
Tight GUI-Business Coupling	Cannot test business logic without GUI
Global Variables (23+)	Hard to track state, not thread-safe
9 Duplicate Search Methods	100+ lines of copy-paste code
Complex Nested Conditionals	4+ nesting levels, hard to understand
Boolean Magic Numbers	<code>ev=1</code> , <code>motor=1</code> unclear intent
Magic Number -1	Silent failures for errors
Inconsistent Inheritance	ElectricCar/Bike don't properly inherit
Poor Variable Naming	<code>slotid</code> , <code>regnum</code> , <code>numOfOccupiedSlots</code> cryptic
No Error Handling	Silent failures, no exceptions

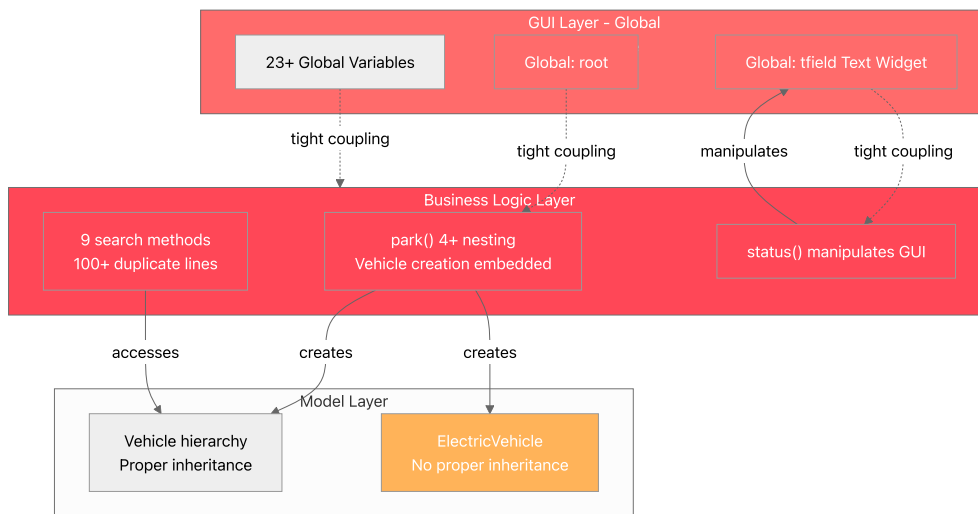
Activity Diagram - park() Method (Original)



Complexity Metrics - Original Design



Dependency and Coupling Issues



Key Problems Summary

High Severity Issues

1. **Tight Coupling:** GUI and business logic intertwined
 - Cannot test business logic independently
 - Cannot reuse logic in different UI
2. **Global State:** 23+ global variables
 - Difficult to track dependencies
 - Not thread-safe
 - Makes code fragile
3. **Duplicate Code:** 9 search methods with identical logic
 - 100+ lines of copy-paste code
 - Maintenance nightmare
 - Bugs need fixing in multiple places
4. **Complex Logic:** park() method has 4+ nesting levels
 - Hard to understand
 - Hard to maintain

- Difficult to test

Medium Severity Issues

5. Boolean Magic Numbers: `ev=1` , `motor=1`

- Unclear intent
- Error-prone
- Self-documenting code impossible

6. Silent Failures: Magic number `-1` for errors

- No indication of failure reason
- Hard to debug
- Poor error handling

7. Inconsistent Inheritance: ElectricCar/Bike don't inherit

- Type checking broken
- Polymorphism not working
- Code smell: improper OOP

8. Poor Naming: `slotid` , `regnum` , `numOfOccupiedSlots`

- Reduces readability
- Faster code rot
- Hard onboarding

9. No Exception Handling

- Silent failures
- Unprofessional error handling
- Poor user experience

Conclusion

The original design shows classic anti-patterns that make the codebase:

- **Monolithic** - Everything in one file
- **Tightly Coupled** - GUI and logic inseparable
- **Untestable** - Cannot test without GUI

- **Duplicated** - Copy-paste code everywhere
- **Complex** - Deep nesting and cryptic logic
- **Not Reusable** - Business logic tied to GUI
- **Hard to Maintain** - Changes needed in multiple places

These issues prompted the need for comprehensive refactoring using design patterns and best practices.